

The Future of Teaching Computer Graphics

Edward Angel

University of New Mexico, USA
angel@cs.unm.edu

Dave Shreiner

Unity Technologies, USA and Sonoma State University,
USA
shreiner@siggraph.org

ACM Reference Format:

Edward Angel and Dave Shreiner. 2024. The Future of Teaching Computer Graphics. In *Special Interest Group on Computer Graphics and Interactive Techniques Conference Educator's Forum (SIGGRAPH Educators Forum '24)*, July 27–August 01, 2024, Denver, CO, USA. ACM, New York, NY, USA, 2 pages. <https://doi.org/10.1145/3641235.3664433>

1 THE FUTURE OF TEACHING COMPUTER GRAPHICS

Computer Graphics has been a standard course in most computer science and engineering programs for at least 40 years. Teaching a basic course in computer graphics has never been easy. The subject combines elements of mathematics, physics, computer science and human perception. An instructor must balance both between theory and practice and teach to a diverse audience of students in computer science, engineering, mathematics, physics, art, and various other disciplines across the campus. In addition, the interested students have a variety of goals ranging from becoming software designers to medical applications to media arts and games.

When computer graphics courses were first widely taught during the 1980s, there was limited hardware available to most students and a lack of standard software. Consequently, the courses taught tended to be bottom-up focusing on algorithms, going from pixels to lines to polygons and their display. Most courses lacked a programming component.

During the 1980s and 1990s, workstations and personal computers made it possible for most programs to offer graphics courses with a programming component. Starting in the 1990s, the OpenGL API became the standard for such courses. OpenGL, and now WebGL, have remained the standard APIs for teaching a first course in computer graphics. Our graphics textbook [Angel and Shreiner, 2020] is now in its eighth edition and uses WebGL. We have learned that a web-based API is a key enabler for both students and instructors in a first graphics course.

A typical course focusses on the graphics pipeline supported by OpenGL/WebGL and GPUs. Students spend much of the course learning to model, manipulate and render three-dimensional objects. A great deal of the time focusses on communicating with and designing shaders. Such courses have been able to cover the fundamentals that include geometry, modelling and rendering that go back to Newman and Sproull [Newman and Sproull, 1973] and Foley and van Dam [Foley and van Dam, 1982] and include a strong

programming component that requires students work with the standard pipeline and to create their own shaders.

Over the past few years, GPUs have grown increasingly powerful and allow for multiple forms of rendering including deferred shading, patch rendering, volume rendering, and ray tracing. New APIs, including WebGPU, Vulkan, DirectX 12, and Metal, provide access to the full power of the GPU but require much more code. An application must not only deal with shaders in the pipeline but must also set up the GPU to exploit its incredible capabilities. Consequently, a “hello world” application as simple as displaying a single triangle can require 100+ lines of code with these newer APIs. The question is can we design a computer graphics course that supports any or all choices of an API, the core topics and some of the recent advances.

We start with the observation that across APIs, the graphics part of the code is similar and is inside the shaders. The large amount of code necessary for basic applications using WebGPU, Vulkan and other recent APIs, is necessary to setup the GPU but contains very little graphics.

With WebGL, we have been able to hide most of this code in functions we give to our students and only later reveal these innards. Nevertheless, although this approach avoids many of the problems students have at the beginning of the course, problems dealing with code and data working with multiple languages including JavaScript, shader code, and HTML. More importantly, this approach needs modification knowing that our students will have to work in a world of evolving GPUS and APIs.

Suppose we start with just a vertex shader and a fragment shader connected by a rasterizer, i.e., just a simple GPU. We can hard code the data for simple 2-D and 3-D models into the shaders and thus avoid all the details of getting data from a CPU to a GPU. If we provide simple functions for initializing the shaders, whose content can be discussed later, the three functions that correspond to this model are two shaders and a render function, all of which can be contained in a single short HTML file.

By starting with this simple model of a GPU, we can cover the standard core topics—modeling, rasterization, transformations, geometry—in the first half of course with the students doing programming projects that support the traditional basics. This approach avoids many of the problems students have at the beginning of a course. Adding interaction requires peeling back some of what is in the supplied functions but comes in easily. The second half of the course can then focus on a variety of advanced topics over theory and practice.

Many examples of this approach are contained in the course notes for our SIGGRAPH 24 course “An Introduction to Creating Real-Time Interactive Computer Graphics Applications.”

Looking further into the future, we believe that this approach should work with the newer APIs. We have tried it with some basic

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the owner/author(s).

SIGGRAPH Educators Forum '24, July 27–August 01, 2024, Denver, CO, USA

© 2024 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-0517-5/24/07

<https://doi.org/10.1145/3641235.3664433>

examples in WebGPU and found that the shaders and render functions do not require much more additional code. More importantly, the three functions are not very different across APIs. Hence, in a world where students will later be dealing with a plethora of APIs, most of which are still evolving, this approach to a fundamental course in computer graphics should satisfies the needs of most programs and preserve the topics that have been core to the discipline.

REFERENCES

- Edward Angel, and Dave Shreiner. 2020. Interactive Computer Graphics (8th Edition), Pearson Education.
- William Newman, and Robert Sproull. 1973. Principles of Interactive Computer Graphics, McGraw Hill.
- James Foley, and Andries van Dam. 1982. Fundamentals of Interactive Computer Graphics, Addison-Wesley.